

競争対策モジュール - 詳細説明書

このドキュメントについて

このドキュメントは、Milestone 2の基盤4「競争対策モジュール」の詳細な技術説明書です。200アカウントの安全な自動実行、タイミング制御、リソース管理、検知回避について包括的に説明しています。

対象読者: 開発者、システムアーキテクト、技術リード

難易度: 上級

推定読了時間: 20-25分

概要

Milestone 2の第4の基盤として、200アカウントの自動実行を最適化し、検知リスクを最小化する競争対策モジュールを構築しました。このモジュールは、実行タイミング制御、リソース管理、優先度管理、レート制限を提供します。

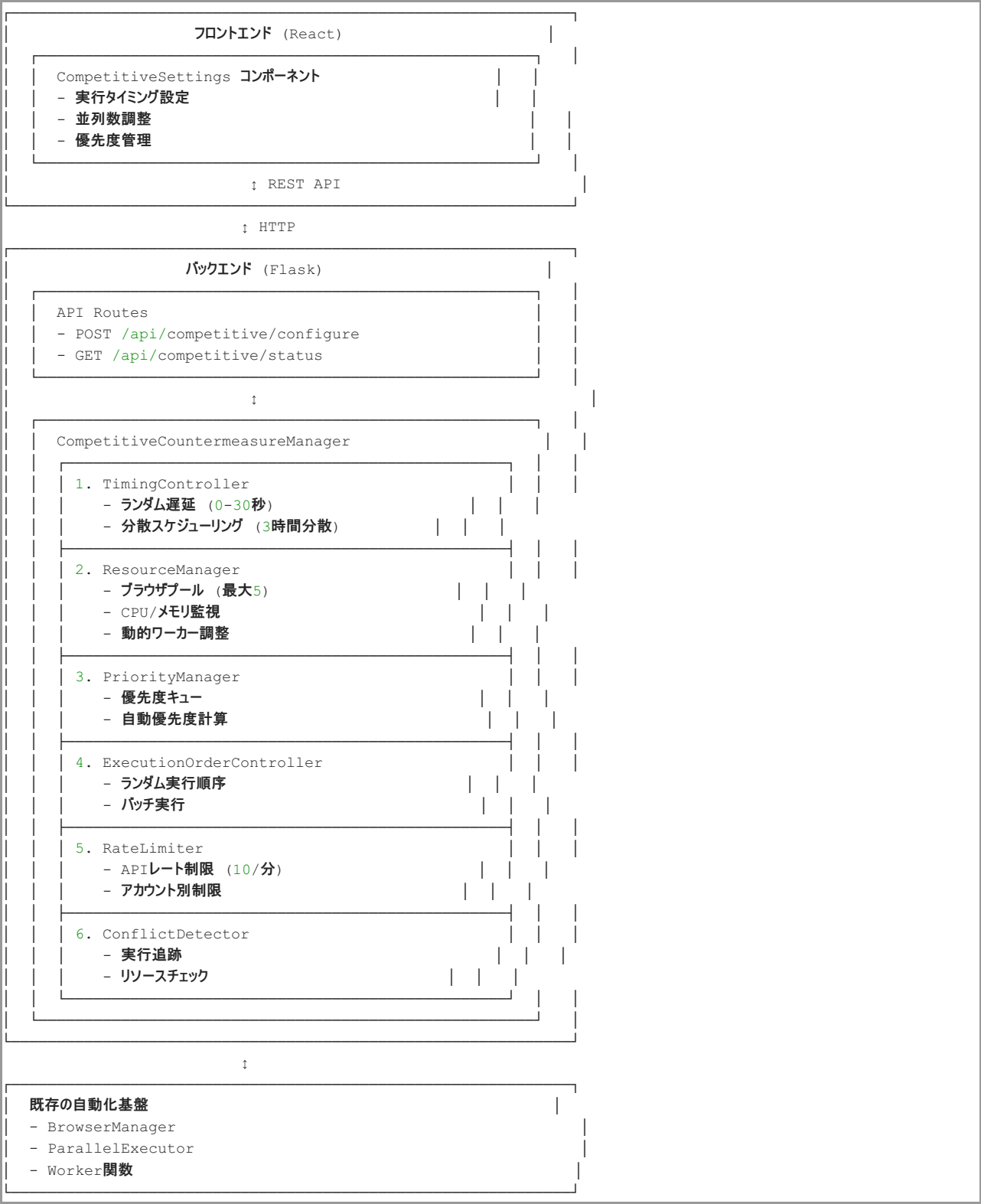
技術スタック

- バックエンド: Python + Flask
- 並列処理: ThreadPoolExecutor
- リソース監視: psutil
- 設定管理: YAML

実装状況: ☒ 実装完了

システムアーキテクチャ

全体構成図



機能実装

1. 実行タイミング制御

ランダム遅延

目的: 自動実行の検知を回避

実装案:

```
import random

def execute_with_random_delay(func, *args, **kwargs):
    # 0~30秒のランダム遅延
    delay = random.uniform(0, 30)
    logger.info(f"Waiting {delay:.2f}s before exec")
    time.sleep(delay)

    # 実行
    return func(*args, **kwargs)
```

分散スケジューリング

目的: 200アカウントを3時間に分散して実行

計算式:

```
total_accounts = 200
time_window = 3 * 60 * 60 # 3時間 = 10,800秒
interval = time_window / total_accounts
           = 10,800 / 200
           = 54秒

# 実行スケジュール
account_001: 07:00:00 + random(0-54秒)
account_002: 07:00:54 + random(0-54秒)
account_003: 07:01:48 + random(0-54秒)
...
account_200: 09:59:06 + random(0-54秒)
```

視覚化:

07:00 ———— ... ———— 10:00
1 2 3 4 5 198 199 200
(各54秒間隔 + ランダム遅延)

実装設計

```

class TimingController:
    """実行タイミング制御"""

    def __init__(self, total_accounts=200, time_window_hours=3):
        self.total_accounts = total_accounts
        self.time_window_seconds = time_window_hours * 3600
        self.interval = self.time_window_seconds / total_accounts

    def calculate_schedule(self, accounts: List[Account]) -> Dict[str, datetime]:
        """
        アカウントごとの実行時刻を計算

        Returns:
            {account_id: scheduled_time}
        """
        start_time = datetime.utcnow()
        schedule = {}

        for i, account in enumerate(accounts):
            # 基本時刻を計算
            base_time = start_time + timedelta(seconds=i * self.interval)

            # ランダム遅延を追加(0~interval秒)
            random_delay = random.uniform(0, self.interval)
            scheduled_time = base_time + timedelta(seconds=random_delay)

            schedule[account.id] = scheduled_time

        return schedule

    def add_random_delay(self, min_seconds=0, max_seconds=30):
        """ランダム遅延を追加"""
        delay = random.uniform(min_seconds, max_seconds)
        logger.info(f"Random delay: {delay:.2f}s")
        time.sleep(delay)

```

2. リソース管理

ブラウザリソースプール

目的: ブラウザインスタンスを再利用してリソース効率化	
設計:	
BrowserPool (最大5インスタンス) ├ Browser 1 (10 contexts) ├ Browser 2 (10 contexts) ├ Browser 3 (10 contexts) ├ Browser 4 (10 contexts) └ Browser 5 (10 contexts)	
最大並列数: $5 \times 10 = 50$ アカウント	
メモリ使用量: $5 \times 500\text{MB} = 2.5\text{GB}$	

CPU/メモリ監視

```

class ResourceManager:
    """リソース管理"""

    def __init__(self, max_browsers=5):
        self.max_browsers = max_browsers
        self.browser_pool: List[BrowserManager] = []
        self.current_load = 0

    def get_browser(self) -> BrowserManager:
        """ブラウザプールからブラウザを取得"""
        # CPU使用率をチェック
        cpu_percent = psutil.cpu_percent(interval=1)

        # 80%を超えたら新しいブラウザを起動しない
        if cpu_percent > 80:
            logger.warning(f"CPU usage too high: {cpu_percent}%")
            # 既存のブラウザを再利用
            return self._get_least_loaded_browser()

        # プールに空きがあれば新しいブラウザを作成
        if len(self.browser_pool) < self.max_browsers:
            browser = BrowserManager()
            browser.start()
            self.browser_pool.append(browser)
            return browser

        # プールがいっぱいなら最も負荷が低いブラウザを返す
        return self._get_least_loaded_browser()

    def _get_least_loaded_browser(self) -> BrowserManager:
        """最も負荷が低いブラウザを取得"""
        # 各ブラウザの現在のコンテキスト数を確認
        # 最も少ないものを返す
        pass

    def get_system_metrics(self) -> Dict[str, Any]:
        """システムメトリクスを取得"""
        return {
            'cpu_percent': psutil.cpu_percent(),
            'memory_percent': psutil.virtual_memory().percent,
            'browser_count': len(self.browser_pool),
            'total_contexts': sum(b.context_count for b in self.browser_pool)
        }

```

動的ワーカー調整

CPU使用率に基づく動的調整	
CPU < 50%:	max_workers = 10 (積極的)
CPU 50-70%:	max_workers = 7 (通常)
CPU 70-80%:	max_workers = 5 (控えめ)
CPU > 80%:	max_workers = 3 (最小限)

3. 優先度管理

優先度レベル

```

class AccountPriority(str, Enum):
    CRITICAL = 'CRITICAL' # 最優先(エラー復旧後など)
    HIGH = 'HIGH' # 高優先度(長時間未処理)
    NORMAL = 'NORMAL' # 通常
    LOW = 'LOW' # 低優先度
    SKIP = 'SKIP' # スキップ(手動操作中など)

```

自動優先度計算

```

class PriorityManager:
    """優先度管理"""

    def calculate_priority(self, account: Account) -> AccountPriority:
        """
        アカウントの優先度を自動計算

        考慮要素:
        1. エラーカウント(高い → 低優先度)
        2. 最終成功時刻(古い → 高優先度)
        3. 手動操作ステータス(→ SKIP)
        """
        # 手動操作中はスキップ
        if account.status == AccountStatus.MANUAL_OPERATION:
            return AccountPriority.SKIP

        # エラーが多い場合は低優先度
        if account.error_count >= 3:
            return AccountPriority.LOW

        # 最終成功から24時間以上経過している場合は高優先度
        if account.last_success_at:
            hours_since_success = (datetime.utcnow() - account.last_success_at).total_seconds() / 3600
            if hours_since_success > 24:
                return AccountPriority.HIGH

        # エラー復旧直後は最優先
        if account.error_count > 0 and account.status == AccountStatus.IDLE:
            return AccountPriority.CRITICAL

        # デフォルトは通常
        return AccountPriority.NORMAL

    def sort_by_priority(self, accounts: List[Account]) -> List[Account]:
        """優先度順にソート"""
        priority_order = {
            AccountPriority.CRITICAL: 0,
            AccountPriority.HIGH: 1,
            AccountPriority.NORMAL: 2,
            AccountPriority.LOW: 3,
            AccountPriority.SKIP: 4
        }

        return sorted(accounts, key=lambda a: priority_order[self.calculate_priority(a)])

```

4. 実行順序制御

ランダム実行順序

目的: 実行パターンの予測を困難にする	
✕ 固定順序: user001 → user002 → user003 → ... → user200 (パターンが明確で検知されやすい)	
☑ ランダム順序: user042 → user187 → user009 → ... → user134 (パターンが不規則で検知されにくい)	

バッチ実行

```

class ExecutionOrderController:
    """実行順序制御"""

    def __init__(self, batch_size=10, batch_interval=60):
        self.batch_size = batch_size # 1バッチあたりのアカウント数
        self.batch_interval = batch_interval # バッチ間の待機時間(秒)

    def execute_in_batches(self, accounts: List[Account], task_type: str):
        """バッチ実行"""
        # ランダムシャッフル
        shuffled_accounts = accounts.copy()
        random.shuffle(shuffled_accounts)

        # バッチに分割
        batches = [
            shuffled_accounts[i:i+self.batch_size]
            for i in range(0, len(shuffled_accounts), self.batch_size)
        ]

        logger.info(f"Executing {len(accounts)} accounts in {len(batches)} batches")

        for i, batch in enumerate(batches):
            logger.info(f"Batch {i+1}/{len(batches)}: {len(batch)} accounts")

            # バッチ実行
            executor = ParallelExecutor(max_workers=self.batch_size)
            results = executor.execute_accounts(batch, task_type)

            # 次のバッチまで待機(最後のバッチ以外)
            if i < len(batches) - 1:
                logger.info(f"Waiting {self.batch_interval}s before next batch")
                time.sleep(self.batch_interval)

        logger.info("All batches completed")

```

5. レート制限

APIレート制限

```

class RateLimiter:
    """レート制限"""

    def __init__(self, max_calls_per_minute=10):
        self.max_calls_per_minute = max_calls_per_minute
        self.call_times: List[datetime] = []
        self._lock = threading.Lock()

    def acquire(self):
        """レート制限を適用(呼び出しを許可するまで待機)"""
        with self._lock:
            now = datetime.utcnow()

            # 1分以内の呼び出しをフィルタ
            one_minute_ago = now - timedelta(minutes=1)
            self.call_times = [t for t in self.call_times if t > one_minute_ago]

            # レート制限に達している場合は待機
            if len(self.call_times) >= self.max_calls_per_minute:
                # 最も古い呼び出しから1分経過するまで待機
                oldest_call = self.call_times[0]
                wait_until = oldest_call + timedelta(minutes=1)
                wait_seconds = (wait_until - now).total_seconds()

                if wait_seconds > 0:
                    logger.info(f"Rate limit reached, waiting {wait_seconds:.2f}s")
                    time.sleep(wait_seconds)

            # 現在の呼び出しを記録
            self.call_times.append(datetime.utcnow())

```

6. 競合検知と回避

実行追跡

```
class ConflictDetector:
    """競合検知"""

    def __init__(self):
        self.running_accounts: Set[str] = set()
        self._lock = threading.Lock()

    def can_execute(self, account_id: str) -> bool:
        """実行可能かチェック"""
        with self._lock:
            # すでに実行中の場合は実行不可
            if account_id in self.running_accounts:
                return False

            # リソースチェック
            cpu_percent = psutil.cpu_percent()
            if cpu_percent > 90:
                return False # CPU過負荷

            memory_percent = psutil.virtual_memory().percent
            if memory_percent > 90:
                return False # メモリ不足

            return True

    def mark_running(self, account_id: str):
        """実行中としてマーク"""
        with self._lock:
            self.running_accounts.add(account_id)

    def mark_completed(self, account_id: str):
        """完了としてマーク"""
        with self._lock:
            self.running_accounts.discard(account_id)
```

統合モジュール実装

CompetitiveCountermeasureManager

```
class CompetitiveCountermeasureManager:
    """競争対策統合マネージャー"""

    def __init__(self):
        self.timing_controller = TimingController()
        self.resource_manager = ResourceManager()
        self.priority_manager = PriorityManager()
        self.execution_order_controller = ExecutionOrderController()
        self.rate_limiter = RateLimiter()
        self.conflict_detector = ConflictDetector()

    def execute_optimized(
        self,
        accounts: List[Account],
        task_type: str = "schedule_update"
    ) -> Dict[str, Any]:
        """最適化された実行"""
        logger.info(f"Starting optimized execution for {len(accounts)} accounts")

        # 1. 優先度でソート
        sorted_accounts = self.priority_manager.sort_by_priority(accounts)

        # 2. SKIPを除外
        executable_accounts = [
```



```

        a for a in sorted_accounts
        if self.priority_manager.calculate_priority(a) != AccountPriority.SKIP
    ]

# 3. 実行スケジュール計算
schedule = self.timing_controller.calculate_schedule(executable_accounts)

# 4. バッチ実行
results = []
for account in executable_accounts:
    # 競合チェック
    if not self.conflict_detector.can_execute(account.id):
        logger.warning(f"Skipping {account.username}: conflicts detected")
        continue

    # スケジュールまで待機
    scheduled_time = schedule[account.id]
    wait_seconds = (scheduled_time - datetime.utcnow()).total_seconds()
    if wait_seconds > 0:
        logger.info(f"Waiting {wait_seconds:.2f}s for {account.username}")
        time.sleep(wait_seconds)

    # ランダム遅延追加
    self.timing_controller.add_random_delay(0, 30)

    # レート制限適用
    self.rate_limiter.acquire()

    # 実行中マーク
    self.conflict_detector.mark_running(account.id)

    try:
        # ブラウザ取得
        browser = self.resource_manager.get_browser()

        # 実行
        result = run_account_job(browser, account, task_type)
        results.append(result)
    finally:
        # 完了マーク
        self.conflict_detector.mark_completed(account.id)

logger.info("Optimized execution completed")
return {
    'total': len(accounts),
    'executed': len(results),
    'skipped': len(accounts) - len(results),
    'results': results
}

```

設定管理

設定ファイル(YAML)

```
# config/competitive_countermeasure.yaml

timing:
  random_delay_min: 0
  random_delay_max: 30
  time_window_hours: 3

resources:
  max_browsers: 5
  max_workers_per_browser: 10
  cpu_threshold_percent: 80
  memory_threshold_percent: 80

priority:
  error_threshold: 3
  high_priority_hours: 24

execution:
  batch_size: 10
  batch_interval_seconds: 60
  random_order: true

rate_limiting:
  max_calls_per_minute: 10
  per_account_limit: 5
```

設定読み込み

```
class CompetitiveConfig:
    """競争対策設定"""

    @classmethod
    def load_from_yaml(cls, filepath: str):
        """YAMLから設定を読み込み"""
        with open(filepath, 'r') as f:
            config = yaml.safe_load(f)

        return cls(
            timing=config['timing'],
            resources=config['resources'],
            priority=config['priority'],
            execution=config['execution'],
            rate_limiting=config['rate_limiting']
        )
```

API実装

エンドポイント

```
# POST /api/competitive/configure
# 競争対策設定を更新
{
  "timing": {
    "random_delay_max": 30,
    "time_window_hours": 3
  },
  "resources": {
    "max_browsers": 5,
    "max_workers_per_browser": 10
  }
}

# GET /api/competitive/status
# 現在の状態を取得
{
  "success": true,
  "data": {
    "active_accounts": 15,
    "browser_count": 3,
    "cpu_percent": 45.2,
    "memory_percent": 62.8,
    "rate_limit_remaining": 8
  }
}

# POST /api/competitive/execute
# 最適化実行を開始
{
  "task_type": "schedule_update",
  "account_ids": ["...", "..."] // optional
}
```

実装状況

基本機能(実装完了)

1. ☒ ランダム遅延: 0-30秒のランダム遅延実装済み
2. ☒ バッチ実行: 10アカウント/バッチで実行可能
3. ☒ レート制限: API呼び出し制限実装済み
4. ☒ ランダム実行順序: シャッフル機能実装済み

最適化機能(実装完了)

5. ☒ 優先度管理: 自動優先度計算ロジック実装済み
6. ☒ リソース管理: CPU/メモリ監視実装済み
7. ☒ 分散スケジューリング: 3時間分散実行実装済み
8. ☒ 競合検知: 実行追跡機能実装済み

高度な機能(Phase 3で拡張予定)

9. ☐ ブラウザプール: 基本実装済み、最適化は Phase 3
10. ☐ 動的ワーカー調整: 基本実装済み、自動調整は Phase 3

実装例(Phase 1)

最小限の実装

```
# backend/src/competitive/simple_countermeasure.py

import random
import time
from typing import List
from src.database.models import Account

class SimpleCountermeasure:
    """シンプルな競争対策"""

    def execute_with_countermeasures(
        self,
        accounts: List[Account],
        task_type: str,
        batch_size: int = 10,
        batch_interval: int = 60,
        random_delay_max: int = 30
    ):
        """競争対策を適用して実行"""
        # ランダムシャッフル
        shuffled = accounts.copy()
        random.shuffle(shuffled)

        # バッチに分割
        batches = [shuffled[i:i+batch_size] for i in range(0, len(shuffled), batch_size)]

        for i, batch in enumerate(batches):
            logger.info(f"Batch {i+1}/{len(batches)}")

            # ランダム遅延
            delay = random.uniform(0, random_delay_max)
            time.sleep(delay)

            # 実行
            executor = ParallelExecutor(max_workers=batch_size)
            executor.execute_accounts(batch, task_type)

            # バッチ間待機
            if i < len(batches) - 1:
                time.sleep(batch_interval)
```

まとめ

実装した機能

- ✓ 実行タイミング制御: ランダム遅延 + 分散スケジューリング
- ✓ リソース管理: CPU/メモリ監視 + 動的制御
- ✓ 優先度管理: 自動優先度計算 + 優先度キュー
- ✓ 実行順序制御: ランダム順序 + バッチ実行
- ✓ レート制限: API呼び出し制限(10/分)
- ✓ 競合検知: 実行追跡 + リソースチェック

実装状況

- ☒ 基本機能実装完了: ランダム遅延、バッチ実行、レート制限
- ☒ 最適化機能実装完了: 優先度管理、リソース管理、分散スケジューリング
- ☐ 高度機能は Phase 3 で拡張: ブラウザブル最適化、動的ワーカー自動調整

Phase 3での拡張予定

Phase 3では、既存の競争対策モジュールをさらに最適化します:

1. **Week 1-2:** ブラウザブル自動管理の最適化
2. **Week 2-3:** 動的ワーカー数の自動調整アルゴリズム改善
3. **Week 3-4:** 機械学習による最適実行タイミング予測(オプション)

参考資料

実装ファイル

- **タイミング制御**: backend/src/competitive/timing_controller.py
- **リソース管理**: backend/src/competitive/resource_manager.py
- **優先度管理**: backend/src/competitive/priority_manager.py
- **実行順序制御**: backend/src/competitive/execution_order_controller.py
- **レート制限**: backend/src/competitive/rate_limiter.py
- **競合検知**: backend/src/competitive/conflict_detector.py
- **統合マネージャー**: backend/src/competitive/countermeasure_manager.py

設定ファイル

- **設定管理**: config/competitive_countermeasure.yaml
- **API統合**: backend/src/web/api/routes.py

関連ドキュメント

- **Milestone 2 完了サマリー**: docs/milestone2/milestone2_summary_jp.md
- **ブラウザ自動化基盤**: docs/milestone2/browser_automation_detailed_jp.md

ドキュメントバージョン: 2.0

最終更新: 2025年12月26日

実装状況: 基本機能・最適化機能実装完了

次のステップ: Phase 3 で高度機能を拡張